Bakingo PO Demand Forecasting with Meta Prophet — Handout

Author: generated for navajitkumardas@gmail.com **Date:** 2026-06-15 **Branch:** `claude/festive-faraday-ga6m86` **Repo path:** `/home/user/Automation`

---

1. What this project does

Forecasts purchase-order (PO) demand for Bakingo SKUs using **Meta's Prophet** time-series library. The work has three layers:

1. **Build a forecast** for daily demand (qty and value) over a future horizon.
2. **Focus on the top SKUs** — rank by historical revenue, forecast each.
3. **Validate the model** with a 90-day holdout backtest, then tune Prophet configurations and compare error metrics.

All outputs (per-SKU CSVs, forecast plots, metric tables) are committed to the repo under `forecast_out/`.

---

2. The dataset

Property	Value
File	`bakingo_po_20240101_to_20260613.csv`
Rows	36,915 PO line items
Date range	2024-01-01 → 2026-06-12 (894 days)
Unique products	624
Unique stores	16
Total qty	904,926 units

Schema (relevant columns)

Column	Used as
`createdAt`	Time dimension (`ds`) — normalized to a daily date
`productName` , `productId`	SKU identity / filter / ranking key
`qty`	Volume target (`y`)
`totalValue`	Revenue target (`y`) — line-item value in INR

Column	Used as
`fromStoreName`, `fromStoreGstin`	Store dimension (not yet used)

Modeling target

Every model is fit to a **daily aggregated series**: for a given SKU, sum `qty` (or `totalValue`) per calendar day, then reindex so missing days are filled with `0`. That `(ds, y)` frame is what Prophet expects.

---

3. Why Prophet

Prophet decomposes a series into three signals:

```
y(t) = trend(t) + seasonality(t) + holidays(t) + noise
```

It handles: - Missing days and irregular spacing. - Multiple seasonalities (we enable **yearly** and **weekly**). - Holiday effects (we added **India holidays** via `add_country_holidays("IN")`). - Trend changepoints (controlled by `changepoint_prior_scale`, default `0.05`).

It is good for series that look like "trend + repeating pattern + noise", which most cake demand does. Where it struggles is **sparse, bursty, zero-heavy series** — this turned out to be the central issue in our data.

---

4. Scripts produced

Script	What it does	Output dir
`run_prophet.py`	Forecast a single series (default = all-products daily total). Supports `--product`, `--product-id`, `--periods`.	`forecast_out/`
`run_prophet_top.py`	Rank products by `totalValue` (or `qty`) and forecast the top N.	`forecast_out/top/`
`run_prophet_top_both.py`	Same top-N ranking but forecasts **both** `totalValue` and `qty` side-by-side per SKU.	`forecast_out/top_both/`
`run_prophet_backtest.py`	Hold out the last 90 days, train on the rest, compute MAE / RMSE / MAPE / sMAPE / Bias / SumPctErr per SKU per metric.	`forecast_out/backtest/`
`run_prophet_backtest_v2.py`	Same 90-day holdout, but compares **three configurations**: daily-default, daily-tuned-`cps`, weekly aggregation. Picks a winner per SKU per metric.	`forecast_out/backtest_v2/`

How to run

```
# install
pip install prophet pandas matplotlib

# 1) global daily forecast, 90 days ahead
python3 run_prophet.py --periods 90

# 2) top 10 products by revenue, both metrics, 90-day forecast
python3 run_prophet_top_both.py --top 10 --periods 90

# 3) backtest with 90-day holdout
python3 run_prophet_backtest.py --top 10 --holdout 90

# 4) compare daily / daily-tuned / weekly configs
python3 run_prophet_backtest_v2.py --top 10 --holdout 90
```

---

5. Headline forecast — Top 10 SKUs by revenue, 90 days ahead

These use the full history (2024-01-01 → 2026-06-12), daily Prophet, both metrics. Source file: `forecast_out/top_both/_summary_top10_both_90d.csv`.

Rank	SKU	Hist. ₹	Forecast ₹ (90d)	Hist. qty	Forecast qty (90d)
1	Choco Truffle Cake	4,309,145	574,069	32,334	4,125
2	Pineapple Cake	1,603,553	133,659	9,071	1,090
3	Fresh Fruit Cake	1,178,606	185,228	6,732	985
4	Ferrero Rocher Cake	1,011,030	241,048	3,047	755
5	Belgium Choco Mousse	1,001,520	92,101	3,005	360
6	Biscoff Jar Cake	786,736	55,764	7,110	640
7	Dream Cake	620,708	75,349	2,890	471
8	Tiramisu Cake	620,653	69,122	2,928	387
9	Butterscotch Cake	574,629	73,261	5,959	710
10	Choco Vanilla Cake	483,860	55,008	4,223	365

Caveat: these come from the daily, default-`cps` Prophet — the backtest (next section) shows this configuration over-forecasts most cakes.

---

6. Backtest — does Prophet actually predict the last 90 days?

Methodology

1. Reserve the last 90 days of data (2026-03-15 → 2026-06-12) as a **holdout**.
2. Re-train Prophet on every prior day only.
3. Predict the 90-day holdout window.
4. Compare prediction vs. actual using six metrics.

Error metrics (definitions)

Metric	Meaning	Use it for
MAE	mean absolute error (units of `y`)	average day-level miss
RMSE	root mean squared error	penalizes large misses
MAPE	mean abs % error (non-zero days only)	relative day-level miss
sMAPE	symmetric MAPE, safe for zeros	best for sparse series
Bias	mean(forecast − actual)	direction of error (over / under)
SumPctErr	(sum_pred − sum_act) / sum_act × 100	aggregate (90-day total) accuracy

Top-line results (daily, default cps)

Source: `forecast_out/backtest/_metrics_top10_holdout90d.csv`.

SKU	metric	Actual	Forecast	SumPctErr	sMAPE
Choco Truffle	₹	353,469	616,023	+74%	111%
Choco Truffle	qty	2,611	3,588	+37%	118%
Pineapple	₹	118,199	216,611	+83%	114%
Fresh Fruit	₹	117,904	170,722	+45%	117%
Ferrero Rocher	₹	175,206	203,346	**+16%**	134%
Belgium Choco Mousse	₹	59,414	94,125	+58%	127%
Biscoff Jar	₹	23,960	17,095	**−29%**	146%
Dream Cake	₹	67,150	61,562	**−8%**	141%
Tiramisu	₹	58,748	96,896	+65%	123%
Butterscotch	₹	38,161	89,263	+134%	124%
Choco Vanilla	₹	31,695	49,675	+57%	120%

What this tells us

- **Point-level accuracy is poor across the board.** sMAPE ~120% means the model is, on average, missing each daily value by more than the value itself. Reason: most SKUs only sell on ~40-50% of days, and on selling days the qty varies widely. A model that predicts a smooth daily curve can never match this bursty pattern day-by-day.
- **Aggregate accuracy is mixed.** A few SKUs (Dream −8%, Ferrero +16%, Biscoff −29%) are within tolerance for 90-day planning, but most are off by 50%+, with a clear **over-forecast bias** — Prophet is extrapolating an upward trend that didn't continue.

---

7. Backtest v2 — can we do better with tuning or different aggregation?

We compared three configurations on the same 90-day holdout:

Config	Frequency	`changepoint_prior_scale`
A	daily	0.05 (default)
B	daily	tuned per SKU via grid `{0.01, 0.05, 0.1, 0.5}` on a pre-holdout validation window
C	weekly (`W-MON`)	0.05

Source: `forecast_out/backtest_v2/_metrics_top10_holdout90d_compare.csv`.

Point-level error (sMAPE, averaged)

Config	avg sMAPE
A daily default	~125%
B daily tuned	~131%
C weekly	**~52%**

Weekly aggregation **roughly halves** the pointwise error. Bucketing daily PO orders into weeks removes the dominant source of error (zero-vs-spike day-of-week noise) while keeping the trend and seasonality intact.

Aggregate accuracy (|SumPctErr|, lower is better)

Selected examples:

SKU / metric	A	B (cps)	C
Choco Truffle / ₹	74%	**65%** (0.5)	83%
Fresh Fruit / qty	23%	**7%**	23%

SKU / metric	A	B (cps)	C
Ferrero Rocher / qty	13%	16%	**11%**
Belgium Choco Mousse / ₹	58%	**24%**	91%
Biscoff Jar / ₹	29%	**9%**	16%
Dream Cake / qty	1.9%	1.9%	**0.9%**
Tiramisu / ₹	65%	**14%**	128%
Choco Vanilla / ₹	57%	**22%**	81%

- **B (tuned `cps`)** wins on aggregate accuracy for most cakes — a smaller `cps` dampens the upward trend that was causing over-forecasts.
- **C (weekly)** wins on shape — the only config with usable point-level accuracy, but it sometimes amplifies the over-forecast bias on the aggregate sum because we lose the ability to fit Indian holiday spikes at weekly granularity.

Special case

Butterscotch Cake is unfit by every configuration (+96 to +142% over-forecast). This is not a modeling failure — the last 90 days diverged from history in a way that looks like a real demand shift (new SKU launched, channel change, or pricing change). No purely historical model can catch a structural break that occurs at the boundary of the training data.

---

8. Recommendations

Choose the right config for the job

Use case	Recommended config	Why
Weekly replenishment / production planning	**C — weekly Prophet**	sMAPE 27-90%, usable shape; works for both ₹ and qty
90-day budget / category totals	**B — daily, per-SKU tuned `cps`**	Best aggregate accuracy on 7 of 10 SKUs
Day-of forecasting (next-day order)	**None of the above — not reliable**	Daily PO data is too sparse; consider intermittent-demand models (Croston, ADIDA, or zero-inflated GLM)

Improvements worth trying next

1. **Aggregate to store × SKU × week** instead of all-stores × SKU × day — one big source of noise is that different stores order on different days; combining them masks store-level cadence. Per-store may model each store's weekly reorder cycle.
2. **Add `add_regressor` features** for known drivers: promo flags, holiday eve indicators (Rakhi, Diwali, Christmas, Valentine's), city COVID/festival lockdowns.

3. **Switch model for "packaging" SKUs** (Sticker Roll, Carry Bag, etc.). These have ~5% nonzero days — Prophet is wrong tool. Use Croston's method or simple moving average per-store.
4. **Investigate the Butterscotch break-point.** If it's a real shift, set a manual changepoint at the inflection date, or trim training to only the post-shift data.
5. **Rolling-origin cross-validation** with Prophet's built-in `cross_validation()` would give a more robust accuracy estimate than the single 90-day holdout — recommended before publishing forecasts.

Operating principle

Forecasts at this level of accuracy (~20% on quarterly totals for the best SKUs) are useful for **strategic** planning — quarterly budgets, capacity sizing, top-line revenue projections — but should **not** drive day-level operations like daily kitchen production schedules. For that, you need either much higher-frequency data (hourly orders), or richer features (promo calendar, weather, last week's sell-through), or a different model class.

---

9. Repo layout

```
/home/user/Automation/
├── run_prophet.py                  # generic single-series forecast
├── run_prophet_top.py              # top-N forecast (one metric)
├── run_prophet_top_both.py         # top-N forecast (both metrics side-by-side)
├── run_prophet_backtest.py         # 90-day holdout, metrics, per-SKU plots
├── run_prophet_backtest_v2.py      # 3-config comparison + winners
├── HANDOUT.md                      # this document
└── forecast_out/
    ├── forecast_ALL_PRODUCTS.{png,csv}      # global daily total
    ├── components_ALL_PRODUCTS.png
    ├── top/                                  # top-N (one metric)
    ├── top_both/                             # top-N (both metrics)
    ├── backtest/                             # baseline backtest
    └── backtest_v2/                          # config comparison
```

Every per-SKU CSV in `forecast_out/` carries the columns `ds, yhat, yhat_lower, yhat_upper` over the full prediction window (history + future). Holdout CSVs additionally carry `y` (actual).

---

10. Glossary

- **PO** — purchase order; each row in the input CSV is one product line on one PO.
- **`ds`** / **`y`** — Prophet's required column names: timestamp and target.
- **Changepoint** — a date where the underlying trend rate changes.
- **`changepoint_prior_scale`** (**`cps`**) — controls how flexible the trend is. Higher = more changepoints = more responsive to recent data but overfits noise. Lower = smoother / more conservative.

- **Holdout** — slice of recent data hidden from training and used to score the model on its ability to predict the unseen future.
- **sMAPE** — symmetric mean absolute percentage error. Bounded in [0%, 200%], unlike MAPE it doesn't explode when actuals are zero, which is essential for sparse demand data.